

When the eval metric doesn't fit the architecture: RAGAS vs GraphRAG

Notes from a debugging session on AeroOps GraphRAG, kept as raw material for later writeup.

The headline tension

Manual human review of the GraphRAG pipeline's answers (35-item eval set, scored by hand against ground truth) found roughly 26/35 fully correct, 6 partial, 3 wrong. Call it ~80% full-or-partial correctness. Multi-hop reasoning chains traced correctly with right citations. Fan-out queries over LEADS_TO edges worked. Cross-document synthesis worked.

The same 35 items run through RAGAS (faithfulness + context recall, Groq-judged) returned:

Metric	First RAGAS run	After two fixes
Faithfulness (mean)	0.374	0.462
Context recall (mean)	0.408	0.522

A system that a human reviewer would call "mostly working, some precision issues" scored like a system that was barely functioning. That gap is the actual story. Not "GraphRAG is bad" and not "RAGAS is broken" — the gap itself is informative, and figuring out what it was actually measuring took most of an evening.

What was actually wrong: two real bugs, not metric noise

Before blaming the metric, two genuine implementation bugs were found and fixed, and the RAGAS numbers moved as predicted, which is itself evidence the diagnosis was right rather than a hand-wave.

Bug 1 (retriever-side): alphabetical sort destroying rank order before a cap.

`graphretriever_v3.py` had added `(MAX_RECALL_CHUNKS = 12)` to fix an earlier over-recall problem (too many failure modes listed per query). But the chunk-id collector `(flatten_chunk_ids)` built its candidate list as `sorted(set(chunk_ids))` — alphabetical. Chunk ids are named by source document (`D1_c01`, `D3_c06`, etc.), so alphabetical sort meant chunks from document 1 always won the cap regardless of relevance, and chunks from documents 3 onward got silently truncated even when they were the correct answer. Concretely: a question about compressor-surge recovery action needed `D3_c06`; the retriever's already-capped candidate list didn't contain it at all, because eleven `D1_*`

chunks alphabetically outranked it. Fix: collect chunk ids in rank order (the order `_rank_rows` already produced) instead of re-sorting alphabetically, so the cap keeps the most relevant evidence instead of whichever document name sorts first.

Bug 2 (eval-harness side): a separate, independent context cap tuned for a different problem. The RAGAS runner script (`Ragas_eval_groq_stable.py`) had its own `prepare_contexts_for_ragas` step capping context to 6 chunks / 1200 characters each, with an explicit docstring: "Keep RAGAS judge calls small enough to avoid Groq timeouts." A reasonable, locally-correct engineering decision, completely disconnected from what the eval set actually needed. Checking the eval set's own ground truth: one operational scenario needed 12 source chunks to be fully verifiable, another needed 9. Capping at 6 made those items structurally incapable of scoring well no matter how good retrieval was. Proof this was a separate bug from Bug 1: one scenario item used `path_traversal` retrieval (no cap applied there at all) and correctly retrieved all 22 relevant chunks, containing all 9 ground-truth chunks — yet only 6 made it into what RAGAS actually scored against. Fix: raised the default to 15 (covering the eval set's max requirement with headroom), made it tunable via env var for the Groq-timeout tradeoff.

Fixing both bugs moved faithfulness +0.088 and context recall +0.114. Real, validated, in the predicted direction. So a meaningful chunk of the original "abysmal" score was measurement artifact, not system quality. But not all of it — which is where the actual architecture mismatch shows up.

The architecture mismatch, once the bugs were out of the way

Even after both fixes, scores sat around 0.46/0.52 — better, but still far below what the manual review suggested. Digging into specific cases (not the aggregate number) revealed RAGAS's design assumptions don't fit how GraphRAG actually represents and justifies an answer.

1. RAGAS assumes context is a flat bag of text. GraphRAG's context is two channels. The answer generator's prompt has a structured `GRAPH FACTS` block (entities, causes, mitigations, each carrying its own inline chunk-id citation pulled directly from a graph edge's properties) and a separate flat `EVIDENCE CHUNKS` block (the actual fetched text for the union of referenced chunk ids). These can drift apart: a graph fact can correctly cite `[D1_c09]` because that's literally what the edge's `source_chunk_ids` property says, even if `D1_c09`'s actual text got trimmed out of the evidence block by an upstream cap. RAGAS only ever sees the flat evidence-text channel — `retrieved_contexts: List[str]` is its entire interface to "what did the system know." It has no visibility into the structured, pre-verified graph-fact channel at all. So a model can produce a fully graph-grounded citation that RAGAS cannot verify, simply because RAGAS is structurally blind to half of what GraphRAG actually grounds its answers in.

2. Faithfulness checks textual entailment, not graph-verified truth. RAGAS faithfulness asks: is this claim's wording entailed by this chunk's wording? It does not ask: is this claim backed by a verified edge in a knowledge graph? Some of the graph's relationship types — `PART_OF`, `MEASURED_BY`, `IS_A` — are structural domain knowledge with no source chunk at all by design, because they weren't extracted from a passage, they were modeled directly. If an answer ever leans on that kind of fact, there is no chunk text for RAGAS to check it against, faithfulness-wise, regardless of how correct the fact is.

3. Context recall rewards coverage, not path correctness. Recall checks whether the reference answer's claims can be found somewhere in the retrieved context, treated as an unordered set. A system that retrieves the right four chunks but garbles the causal order between them (claims B causes C when the graph says C causes B) scores identically to a system that retrieves the same four chunks and narrates the chain correctly. The entire point of building chain traversal (`retrieve_path`, walking `shortestPath` between two entities and returning ordered nodes plus typed edges) was to get multi-hop reasoning right. RAGAS has no metric surface for "was the chain correct," only "was the vocabulary present."

4. The judge-call cost model assumes naive RAG's typical footprint, not a graph's. Naive RAG retrieval is usually 3 to 10 chunks from a single embedding-similarity pass. GraphRAG multi-hop answers can legitimately need more — the worst case in this eval set needed 12 chunks to justify one operational-scenario answer spanning multiple failure modes and operating factors. RAGAS's per-claim judge-LLM-call pattern gets slow and expensive as context grows, which is exactly why the eval script's author capped it at 6 in the first place. That cap was a rational response to a real cost problem, but it encodes an assumption (small, flat context is normal) that doesn't hold for graph-justified answers.

5. Epistemically honest refusal gets penalized as unfaithful. One case: asked for the source of a specific claim, the model said "there is no information in the provided graph facts that directly supports this." That was the correct response, given what it had actually received (a separate entity-matching bug meant the right node never got retrieved). RAGAS scored it as unfaithful, because it measures "is the answer's content entailed by context," and a correctly-hedged non-answer doesn't entail anything to check. The system was being careful. The metric read that as failure.

The confound: distinguishing "the metric doesn't fit" from "I have a bug"

The harder, more useful part of tonight wasn't identifying the architecture mismatch in the abstract. It was the fact that real bugs (missing entity types in the query router, an "egt" → "egt margin" term expansion injecting an unrelated failure mode into unrelated questions, a hyphen normalization mismatch that silently broke "oil-pressure" from matching "oil pressure") were sitting in the same aggregate low score as the metric-design issues, and they look identical from the outside. A low RAGAS number is consistent with both "your

retriever has a bug" and "the metric can't see what your system actually did right." Telling them apart required going item by item: checking whether the ground-truth source chunk was even present in what got retrieved (a bug, if not), and separately checking whether a present, correct chunk still scored as unfaithful (a metric limitation, if so). Both were true simultaneously in this dataset. The aggregate score doesn't tell you which one you're looking at; only the per-item trace does.

This is itself the most reusable lesson: a bad eval number is a prompt to investigate, not a verdict to report. Some of what it's flagging will be real, fixable bugs (and fixing them should move the number, which is a good sanity check that you understood the bug correctly). Some of it will be the metric's blind spot. Reporting the number without doing this separation is how you end up either falsely confident (when it's secretly hiding a bug) or falsely alarmed (when it's secretly just a metric mismatch).

What a better complementary metric would need

Given the gaps above, a metric suite for graph-grounded RAG probably needs three things RAGAS doesn't provide out of the box:

- **Node/edge recall against a ground-truth graph path.** Since the eval set's ground truth already encodes `ground_truth_nodes` per question, a recall metric over those (did the answer's reasoning touch the entities it needed to) is something RAGAS-style text-entailment can't substitute for.
- **Path-order correctness for multi-hop questions.** Not just "were the right facts mentioned" but "were they connected in the right causal direction." This needs the eval set to encode the expected chain, not just the expected node set.
- **Edge-level citation precision.** Did the answer cite the chunk that the specific claim's graph edge actually points to, not just a chunk that happens to be in the broader context window. This is checkable directly against the graph's own `DERIVED_FROM` / `claim_id` provenance, no judge LLM needed, which would also be much cheaper than RAGAS's per-claim LLM calls.

None of this means RAGAS should be dropped. It's a recognized standard, and reporting it (with the caveats above stated explicitly) is more credible than not using it at all. The honest framing is "RAGAS plus a graph-aware supplement," not "RAGAS replaced by something custom."

Possible Substack angle

This is a natural continuation of the first article's thesis (eval design must come from real user experience, not convenient benchmarks) at one layer deeper: it's not just that benchmark questions can be unrepresentative of real usage, it's that even a well-regarded, standard *metric* encodes unstated assumptions about system architecture, and those

assumptions can be wrong for your architecture specifically while still being completely reasonable for the architecture the metric was designed around (naive, flat-context RAG). The numbers stay correctly computed throughout; what breaks is whether the number means what a reader assumes it means.

A concrete, honest narrative arc: ran RAGAS, got an alarming number, almost reported it as "GraphRAG needs work," instead spent an evening tracing exactly where the number came from, found two real bugs (fixed them, numbers moved as predicted, confirming the diagnosis) and three architectural blind spots in the metric itself (context-channel separation, textual entailment vs graph provenance, path-order blindness), and ended up with a more defensible, more specific claim about what the system does well and where the standard metric can't see it. That arc, told plainly with the actual before/after numbers, is a stronger piece than either "here's my eval score" or "RAGAS is flawed" alone.

Open threads, not yet resolved

- `_rank_rows` in the retriever scores candidate failure modes partly by how richly documented they are (count of symptoms, causes, mitigations, leads_to edges), not purely by query relevance. This systematically over-surfaces well-documented nodes (compressor surge, in this corpus) whenever entity matching is weak. Not yet fixed; flagged during tonight's session as a likely contributor to noisy `query_plan_recall_heavy` retrieval, separate from the two bugs above.
- The RAGAS context cap is still a single global number (now 15) regardless of retrieval type. `path_traversal` results are already precise and arguably shouldn't be capped at all, while `query_plan_recall_heavy` results may still benefit from a tighter budget. A retrieval-type-aware cap is the more correct fix, not yet implemented.
- Today's `query_understanding_v3.py` fixes (missing entity types, blind term expansions, hyphen normalization) were verified with deterministic Python tests, not yet verified end-to-end through a live answer generation pass, since Groq credits ran out before that could be tested. First thing to confirm once credits reset.